

Optimus - Test Strategy

Version 1.8 — reviewed local filing candidate, 2026-10-04.

Predecessor: Optimus-Cost-Agent-Test-Strategy-v1.7.pdf; SHA-256

31a60c6a198c60cc1203ff5c4a8e6e0300a820ec18cc702e25f246edc51dc0b0. Every original page position remains represented once. Affected bodies are replaced in place; unrelated content/diagrams are carried unchanged. New Plan 12 sections are appended. The source manifest maps original positions to physical pages.

Prepared by Codex as architect/reviewer. Claude files and verifies under the existing offline local-filing release. This document grants no paid/live calls, activation, push, PR or merge authority. Historical editions and their acceptance remain distinct.

Observed offline baseline: cf9bb9d, clean. V1 capacity outcomes/meter, V3 shared remaining-allowance preflight, Task 11 product/evaluation separation, named trusted test composition, per-package coverage and Task 12 boundaries pass offline verification. Windows 6830 passed; WSL 6641 passed; both exits 0, with skips/deselections retained in closing evidence.

Production holds: ENFORCEMENT_ACTIVE=False; trusted_snapshot() returns None; shipped claude-haiku resolves to claude-haiku-4.5. Normal production context enforcement/default removal remain held. Positive test composition uses a synthetic eligible profile. Luna remains ineligible and Qwen unqualified; genuine quality and real-editor Task 13 remain UNRUN.

Client-supplied ACP mcpServers remain local with MCPTrustRegistry descriptors/approval. Agent/Gateway stay separate processes in the same loopback network namespace; no Gateway MCP broker is added.

Companion documents: Architecture/HLD v2.19, LLD v2.42, Guardrails v1.4, Test Strategy v1.8. The numeric set follows the reported human offline release. Its verbatim D7 decision still needs saved transcript provenance under ADR-011 at filing; no quotation is fabricated here. Documentation filing alone does not close CP4 or dispose of live requirements.

This is a complete reviewed local filing candidate. It is not a claim that authoritative predecessors have already been replaced or that publication/operational acceptance occurred.

1. Test objectives

The strategy proves that a developer can complete full Plan and Agent runs with only OPTIMUS_GATEWAY_URL and OPTIMUS_API_KEY in the agent process. No upstream credential is resolvable by the agent. Separately, the loopback Gateway receives exactly its approved aggregator credential and never exposes it.

2. Scope and non-scope

In scope:

- strict loopback URL and bind enforcement, including WSL2 same-network-namespaces topology;
- OpenRouter default and bounded Vercel option through the OpenAI-compatible transport;
- deterministic search live compatibility and plugin deprecation gate;
- include/exclude domain enforcement and returned-URL revalidation;
- independent package/OSV routes without search configuration;
- bounded extract provenance, redirect, SSRF, media, size, and time controls;
- provider-reported usage/cost authority and full request attribution;
- OTel/OTLP export to a real Phoenix tier;
- separate agent and Gateway credential/egress scans.

Out of scope:

- hosted staging Gateway, OAuth/device flow, tenants, org/project wallets, Vault;
- direct provider adapters;
- cross-run spend policy (P9 . 85 - FU - 3);
- MCP Gateway endpoint or contract;
- engine/model comparison matrices.

3. Test pyramid and evidence tiers

Tier	Dependency	Fakes permitted	Claim supported
Unit	In-process functions	Yes	Local logic and contracts
Contract	Recorded request/response shape	Yes	Parser and schema behavior
requires_redis	Real TimeSeries-capable Redis	No	Persistence and retention
requires_gateway	Real loopback Gateway + approved credential	No	Gateway/provider behavior
ACP protocol	Independent acpx client	No	Real client compatibility
E2E	Spawned ACP process + named dependencies	No	Golden workflow
Release	Full process/credential/egress evidence	No	Phase 1 sign-off

There is no hosted staging Gateway. Provider fakes remain confined to unit and contract tests and cannot justify a live claim.

Verification ownership

Every design claim maps to an executable unit, integration, E2E, eval, or release-gate check. Coverage enforces the Optimus product group and separate context_engine / optimus_model_policy 80% floors; all-source aggregate is informational under section 8A, and safety-critical modules do not regress.

LLD Section	Design Claim	Test Category (§)	Tier
§4A — State Machine	AwaitingApproval gate cannot be bypassed	§6 — Mode/State	Unit + Integration
§6 — Gateway Layer	POST /v1/responses uses input field, not messages	§8 — Gateway	Unit + Integration
§6 — Gateway Layer	No provider key present in local environment during run	§8 — Gateway	E2E (Release Gate)
§9 — Tool Policy	EvidenceRequest.query sent verbatim (not substituted)	§7 — Tools	Unit
§9 — Tool Policy	EvidenceExtractRequest rejects out-of-set URLs	§7 — Tools	Unit
§9 — Evidence Ledger	total_cost_usd() reconciles against gateway response	§9 — Cost	Unit + Integration
§9D — Gateway Policy	Gateway revalidates domain allowlist independently	§8 — Gateway	Integration
§10 — Telemetry	RedisTimeSeries TS.CREATE idempotent with TS.ALTER fallback	§10 — Telemetry	Integration
§11 — Sprint 1 Gate	One-key setup: full run with only OPTIMUS_* env vars	§12 — Release Gate	E2E

5. Mode and State Transition Tests

LLD reference: §4 Behavioral Governance, §4A Agent State Model. These tests prove the state machine is deterministic and that no path bypasses the mode enforcement primitives.

Unit Tests — assert_mutation_allowed()

- [] In Plan/Chat mode: calling assert_mutation_allowed() raises Code -32002 with message "mutation forbidden in Plan/Chat mode".
- [] In Agent mode before AwaitingApproval: assert_mutation_allowed() raises Code -32002.
- [] In Agent mode after approval granted: assert_mutation_allowed() returns None (no exception).
- [] Calling any mutation tool (write_file, shell_exec, shadow_apply) in Plan/Chat mode raises -32002 before any I/O occurs.

Unit Tests — State Transitions

- [] Idle -> Planning: valid on any user request.
- [] PlanReady -> Executing (direct): rejected with -32002; must pass through AwaitingApproval.
- [] AwaitingApproval -> ChatOnly on timeout: confirmed after configurable timeout_ms expires.
- [] Failed -> Planning: retry counter increments; failure context injected into next prompt payload.
- [] Failed -> Terminated: at the configured finite work boundary, state transitions truthfully and a stop report is emitted. Pin the current three planning rounds, 30 minutes and two repeated failures separately.

Integration Tests — Mode Boundary

- [] Full Plan/Chat run: agent completes discussion and returns plan text; no file in working tree is modified; telemetry records 0 mutation calls.

6. Tool Invocation Tests

- Search is authorized only by existing policy signals and the harness gate.
- The Gateway forwards the effective include/exclude policy and rejects off-policy citation URLs.
- At `max_tokens=16`, the live deterministic plugin returns typed URL annotations or fails.
- URL, title, and non-empty content are required; annotation offsets are ignored.
- Extract requires exact prior Gateway-recorded provenance and passes redirect/SSRF/media/size/time bounds.
- Package and OSV routes remain callable without a Tavily/search credential.
- Search, extract, package, and advisory usage envelopes reconcile independently.

7. OptimusGatewaySettings Unit Tests

- Default `http://127.0.0.1:8765` passes.
- `localhost` and canonical IPv6 loopback pass.
- Non-loopback DNS names and IP literals fail closed.
- URI `userinfo`, ambiguous host parsing, and non-HTTP(S) schemes fail closed.
- No field or environment variable authorizes non-loopback.
- `production_mode`, extra origins, hosted origins, and signed tenant profiles are absent.
- `optimus_api_key` is masked in repr, serialization, logs, telemetry, and state.

7.1 Integration, E2E, and egress gates

- Direct policy-violation tests target the real loopback Gateway.
- Aggregator routing/fallback evidence records the returned provider and model.
- The agent and ACP child may egress only to the loopback Gateway.
- The Gateway may egress only to the configured aggregator, approved package/OSV hosts, approved extract targets, Redis, and configured OTLP endpoint.
- Agent and Gateway environments are scanned separately.
- The agent has no upstream key; the Gateway has the approved aggregator key by design.
- After replacement acceptance, the Gateway has no direct-provider, Tavily, or LangSmith key.
- Crafted tool output cannot cause direct agent egress.
- WSL2 evidence proves both processes share one network namespace.

Failure behavior

Malformed or missing successful provider usage/cost remains invalid; the stage records explicit unknown cost and preserves known subtotals without latching the product session closed. Permanent authentication, policy, schema, and validation failures do not retry. Transient faults have a bounded retry budget and record every attempt.

For enforced model requests, the following bounded retry policy supersedes generic transient-retry examples: The legacy host permits an initial attempt plus at most three retries. The inactive-policy Gateway separately permits at most three upstream attempts, including its legacy transient timeout/URL/5xx retries. Once enforcement is active, the Gateway permits one primary attempt and at most one identical-payload resend on the same route, only after a definitely NOT_SENT result (including a proven pre-connection timeout) or rejected HTTP 429. A timeout after connection, uncertain delivery, 5xx or unknown accounting is not resent. The host honors non-retryable classifications so it cannot multiply attempts. No alternative-model recovery is added. Each physical attempt retains its own receipt.

8A. Coverage policy and shared gate interface

coverage.py with pytest-cov is the canonical line/branch engine. Use one identified full-suite dataset, including unloaded tracked production files from every top-level src/ package, regular or namespace. Focused datasets cannot substitute. The all-source aggregate is informational.

Product group	Exactly-once members	Floor / disposition
Optimus	optimus, optimus_gateway, optimus_security, optimus_model_policy	80% enforced group
Context Engine	context_engine	80% enforced package/group
Evidence Handoff	evidence_handoff, evidence_handoff_runtime	80% informational; enforcement deferred

Independently enforce optimus_model_policy at 80% in addition to its Optimus group. This extra report does not create duplicate ownership. Inventory comes from Git's tracked-file index, not a filesystem walk. Reject missing/duplicate/stale/unmapped groups, top-level src modules with no package ownership, inconsistent coverage sources, missing unloaded files and missing/stale datasets. Native package patterns must not match a similarly named neighbor.

The comparison uses coverage.py/pytest-cov should_fail_under at configured precision 0: round(measured_percentage, 0) < 80 fails. 79.5% compares as 80% and passes; 79.0% fails. Print and retain both measured and compared percentages. This preserves the existing comparison, rather than claiming an unrounded 80% boundary.

Implemented interface at cf9bb9d:

```
tools/check_product_coverage.py --run-full-suite --groups tools/product-coverage-groups.json --report-  
json <report.json>  
tools/check_product_coverage.py --data-file <full-run-data> --run-record <report.run.json> --groups  
tools/product-coverage-groups.json --report-json <report.json>
```

Exactly one mode is required. Run mode invokes sys.executable -m pytest once with --cov --cov-branch --cov-fail-under=0 --cov-report=term-missing -q, preserving repository addopts/run-context and the true pytest exit. No nested uv or narrowed selection. It writes <report>.run.json with schema, HEAD, dirty flag, fixed command, times, true exit and dataset SHA-256. Report mode never reruns tests and rejects a dataset that does not match its record hash. A nonzero suite exit fails the gate even when floors pass. Required floor failures also fail; informational rows retain BELOW but do not fail the wrapper. The wrapper calls coverage's reporting/comparison API, not additional coverage CLI subprocesses; therefore no per-row process exit is claimed.

The sidecar is provenance for a trusted recorded run, not an authenticated artifact, a dirty-tree digest or a complete configuration snapshot. Reuse only with the corresponding reviewed full-run log/run context and unchanged source/configuration. Verify those externally when accepting a sealed dataset; do not infer full selection or changed-configuration validity from the hash alone. The current final clean-candidate datasets were independently checked in evidence-verification.json.

Until the profiles lane lands, the hook entry is unchanged and keeps pyproject fail_under=80 (an Optimus-only interim floor). Only CI switches to --run-full-suite. The profiles lane later moves the hook to commit with no coverage gate. The released final CP4 gate also uses run mode. Retain hook ID optimus-pytest-coverage, optimus-check: pytest-coverage and existing ADR-016 skip custody; no new ad hoc waiver. Shared source-scope custody remains HARDENING-ITEM-COVERAGE-SOURCE-SCOPE.

CI's rows survive in the step log. JSON report upload is not implemented and is not a claimed retained artifact. No new Evidence Handoff tests are required for informational coverage.

8B. Observability test strategy

Tool	Category	Purpose	Counts toward code coverage
coverage.py / pytest-cov	Code coverage	Production-code execution	Yes
DeepEval / Ragas	LLM quality	Correctness and groundedness	No
PyRIT	Adversarial	Injection and tool-control safety	No
OpenTelemetry / OTLP	Trace observability	Vendor-neutral span contract/export	No

Run the real Phoenix evidence tier and assert required span attributes and request correlation, parent/child relationships, secret redaction, batching/retry outcome, validation/policy disposition and final failure/success disposition. A missing real collector/exporter makes that claim UNRUN.

Phoenix is the documented default, not an API dependency. No LangSmith dependency, endpoint, credential or amortized per-request charge exists. These observability claims are separate from §8A's source coverage gates.

7A. Deterministic Search Compatibility Gate

The deterministic search compatibility gate uses one cheap Flash/Haiku-class model and one default engine request shape. The approved 2026-07-27 baseline is:

- max_tokens=16;
- 3 annotations on the minimal-output probe;
- 3/3 include-domain results allowed and 0 violations;
- 0/3 excluded-domain results forbidden and 0 violations;
- 9/9 annotations with HTTPS URL, title, and non-empty content;
- mean latency 7,068.7 ms/search;
- provider-reported mean cost \$0.0051584/search;
- direct extract 270,008 bytes -> 60,077 text characters in 589.5 ms.

These are evidence values, not permanent performance thresholds. Each run reports fresh values and fails on missing search execution, annotations, domain enforcement, or provider usage/cost. Engine and model comparison matrices are out of scope. A verified-or-fail server-tool successor requires a separate live test proving max_uses: 1, exactly one search, search-use accounting, valid annotations, domain enforcement, and fail-closed behavior when execution evidence is absent.

9. Error, Retry, and Failure Injection Tests

LLD reference: §6 Resilient Provider Calling Layer, §4A Failure and Retry Lifecycle. These tests prove that transient failures are retried with backoff and permanent failures abort cleanly without side effects.

Unit Tests - RetryPolicy

- Current RetryPolicy: initial call plus up to three retries; pin third-attempt success and escalation after the fourth attempt. Under enabled Gateway policy, pin a maximum of two physical attempts only after not-sent/429 with identical payload/route and host non-retryable propagation.
- HTTP 429 under enforced policy: at most one identical payload resend on the same route; all physical attempts count.
- PermanentGatewayError: ABORT_WITH_REPORT returned immediately; no retry; failure report emitted.
- PolicyViolationError: ABORT_WITH_REPORT; escalation signal emitted.
- Independent evaluation budget exhaustion stops that evaluation and preserves receipts. A successful product response above the former \$0.05 is delivered; no product dollar stop remains after migration.
- Finite configured attempts exhausted: report the true failure and all attempt identities; no retry-until-pass.

Integration Tests - Failure Injection

- The legacy host permits an initial attempt plus at most three retries. The inactive-policy Gateway separately permits at most three upstream attempts, including its legacy transient timeout/URL/5xx retries. Once enforcement is active, the Gateway permits one primary attempt and at most one identical-payload resend on the same route, only after a definitely NOT_SENT result (including a proven pre-connection timeout) or rejected HTTP 429. A timeout after connection, uncertain delivery, 5xx or unknown accounting is not resent. The host honors non-retryable classifications so it cannot multiply attempts. No alternative-model recovery is added. Each physical attempt retains its own receipt. Tests cover third-attempt success and escalation after the fourth attempt in the legacy policy, and separately the enabled Gateway maximum of two physical attempts with host non-retryable propagation.
- Fitness failures use configured planning/time/repeated-failure boundaries; targeted replan context is supplied only while those bounds permit continuation. Pin the actual stop and effect evidence.
- Evaluation cap exhaustion stops only its approved evaluation. Product count/time/cancellation/permission failures preserve truthful telemetry and effect state; do not relabel them as dollar exhaustion.

10. Schema Validation Tests

LLD reference: §1 ACP Protocol Framing, §9 Tool Registry. These tests prove malformed inputs are rejected before processing and Pydantic v2 validators enforce invariants at the boundary.

Unit Tests - ACP Framing

- Content-Length: 0 with non-empty body: rejected with -32700 Parse Error.
- Content-Length > MAX_HEADER_SIZE: rejected with -32600 Invalid Request.
- Non-numeric Content-Length: rejected with -32700 Parse Error.
- Body larger than declared Content-Length: truncated; remaining bytes are not leaked into the next message.
- JSON-RPC body missing method: rejected with -32600 Invalid Request.
- JSON-RPC body with unknown method: rejected with -32601 Method Not Found.

Unit Tests - Pydantic Models

- EvidenceRequest with an empty query string: ValidationError before any Gateway call.
- EvidenceExtractRequest with max_chars_per_source=0: ValidationError.
- OptimusGatewaySettings with empty optimus_api_key: ValidationError.
- GatewayUsage with negative billing_units: ValidationError.
- GatewayUsage with negative cost_usd: ValidationError.

11. Security and Trust Boundary Tests

- Secret scans include the aggregator key and every retired direct-provider, Tavily, and LangSmith key name.
- Strict-loopback validation replaces production-mode permutations.
- URL parsing rejects userinfo, non-HTTP(S), non-loopback hosts, ambiguous IP forms, and DNS rebinding seams.
- Extract revalidates every redirect and final address and blocks private/link-local/loopback resolution.
- Tool output is untrusted and cannot become policy or executable content.
- Provider-reported cost must be Decimal-parseable, non-negative, and attributable.
- Logs and traces preserve field names while redacting secrets.

Golden task evidence

Golden tasks run against the real local Gateway. Named live claims use real dependencies. ACP protocol evidence uses acpx, not a project-authored client.

12. Golden Task Regression Suite

Every golden task records:

- expected mode and final state;
- expected tool class and authorization outcome;
- cost band and provider-reported usage;
- mutation behavior;
- Gateway and provider request identity;
- trace identity and final disposition.

The suite covers Plan-only refusal, approved Agent mutation, deterministic search, bounded extract, free package/advisory routes, retry exhaustion, malformed usage/cost, and credential leakage attempts.

OTel/OTLP-to-Phoenix evidence replaces LangSmith assertions. A trace failure cannot silently count as success; delivery state and final disposition must be explicit.

Process-scope assertion

The agent has no upstream credential. The Gateway has exactly the approved aggregator credential and never exposes it through logs, telemetry, state, responses, child environments, or error text.

13. Phase 1 Release Gates

Transport and protocol

- Shadow workspace, mutation guard, and independent ACP-client evidence pass.
- Strict-loopback topology and WSL2 namespace evidence pass.
- Both agent-facing completion shapes validate; mixed shapes fail.

Runtime and credentials

- Full Plan and Agent runs use only the two agent-facing variables.
- Agent and child processes have no direct external egress.
- Gateway has one approved aggregator credential and no retired keys after acceptance.

Evidence and cost

- Search plugin compatibility/deprecation gate passes.
- Package and OSV routes work without search configuration.
- Extract provenance and SSRF gates pass.
- Provider-reported usage/cost reconciles across response, ledger, and TimeSeries.

Observability

- Real OTLP spans reach Phoenix with required fields and redaction.
- No LangSmith dependency or amortized charge exists.

Release sign-off remains governed by the authoritative Plan 9.6 live-verification gate.

14. Guardrail & Workflow Test Cases

LLD reference: §12 (Guardrail & Workflow Component Contracts). Companion reference: Agent Execution Guardrails & Workflow Strategy v1.4 §11. Consistent with Objective 1 (§1), each control below traces to at least one executable test category; a control without a category is not considered validated for Phase 1.

14.1 Permission policy tests — deny precedence over allow; mode short-circuit; impact-class HOLD; classifier cannot overturn a deny.

14.2 Shell validator tests — destructive, pipe-to-shell, environment/credential access, ANSI, insecure-transport, and egress patterns all BLOCK before any subprocess is spawned.

14.3 Unicode / homoglyph tests — Cyrillic-vs-Latin confusables in hostnames and paths (e.g. U+0456 vs U+0069) are detected and held/blocked.

14.4 Prompt-injection fixture tests prove poisoned agent config and poisoned MCP tool metadata are caught by ConfigTrustScanner / MCPTrustRegistry on ingest.

14.5 MCP autoload denial tests — a server bundled in a cloned repo does not auto-load; a manifest-hash change forces re-approval; allowed_tools are enforced.

14.6 Pre-commit / CI parity tests — the same rule set (Ruff, Bandit, AST-grep, hygiene hooks) fails identically locally and in a clean CI checkout.

14.7 Bypass tests — --no-verify, force-push to main, unsafe .env reads, and unsafe network commands are all blocked.

14.8 Loop control tests — the loop stops on completion, on max_iterations, on wall-clock exhaustion and repeated-failure detection; it never bypasses §14.1/§14.2 enforcement.

14.9 Skill loading & trust tests — a skill loads only when matched; an untrusted/draft skill is blocked; declared allowed_tools are enforced; a skill cannot override project/user deny rules.

14.10 Additions to the §4 Requirements-to-Test Traceability Matrix

LLD Section	Design Claim	Test Category (§)	Tier
§12A — Permission	Deny precedes allow; classifier cannot overturn deny	§14 — Guardrails	Unit
§12A — Pre-Tool Guard	Bad tool call blocked before execution	§14 — Guardrails	Unit + Integration
§12A — Shell Validator	Homoglyph / pipe-to-shell / egress blocked pre-spawn	§14 — Guardrails	Unit
§12B — MCP Trust	No auto-load from cloned repo; hash change re-approves	§14 — Guardrails	Unit + Integration
§12B — Config Scan	Poisoned config / tool metadata caught on ingest	§14 — Guardrails	Unit
§6 / §12 — CI Parity	Local and clean-CI checks fail identically	§14 — Guardrails	Integration

LLD Section	Design claim	Test category	Tier
§12C — Goal Loop	Stops on completion / max-iteration / wall-clock / repeated failure	§14 — Guardrails	Unit + Integration
§12D — Skills	Match-only load; draft blocked; allowed-tools enforced	§14 — Guardrails	Unit

Release-gate note: §13 (Phase 1 Release Gates) retains the release-blocking no-guardrail-bypass check covering §14.7. These continuation rows belong to §14.10's requirements traceability matrix.

15. Plan 12 claim-to-evidence strategy

15.1 Coverage and checkpoint evidence

Section 8A defines one complete coverage dataset, independent product reports and separate 80% floors for `context_engine` and `optimus_model_policy`. The source-group inventory must cover every top-level import package under `src/` exactly once, including informational groups. A new package, duplicate membership or unassigned package fails the guard rather than disappearing from aggregate measurement. The top-level model-policy package floor is an extra report, not a second product ownership.

Historical candidate/test/coverage seals and post-run deltas belong in the closure records. They do not prove changed executable bytes, route quality, persistent enforcement or live editor behavior.

The three historical strict xfails are resolved at cf9bb9d: their assertions now pass and the markers are removed. This does not turn skipped, deselected or UNRUN evidence into passes. Skips, deselections, failed/invalidated attempts and unrun marked tiers remain explicit. Later testing may measure more files and reduce percentages. A `tail/wrapper` exit never substitutes for the underlying coverage/test exit.

Use existing focused checks for accepted-value assertions and documentation-only fixes. Do not repeat the full suite simply for numeric test additions or provenance wording. A newly released substantive product migration requires its applicable checkpoint verification; do not claim the earlier full run tested those new bytes. Keep this within CP4 rather than inventing per-task review gates.

15.2 Offline claims and required cases

Claim	Required cases / source scopes	Tier
Extractability	Forbidden imports; independent noneditable wheel import; packaged YAML; negative tainted extraction probe	Unit + isolated wheel
Exact authority	All-turn outcomes/effects/approvals preserved when ordinary history is omitted; model text cannot manufacture facts	Unit + host integration
Strategies differ	Hybrid anchor/tail discriminator, deduplication, complete newest-turn sliding, zero sliding maintenance	Unit
Packing safety	ASCII/code/CJK/emoji; exact/one-over token and byte input; maximal fresh/reused summary; non-subadditive negative backstop; preflight zero-call rejection	Unit
Checkpoint integrity	Prefix append/change/reorder, stale configuration/revision, cancellation, malformed/oversized/partial output, no publication on rejection	Unit + host integration
Refusal/fallback	Every known and unknown reason, unavailable zero-provider case, failed-summary receipt retained, $\leq$ floor full fallback, $>$ floor OPEN refusal and healthy recovery	Unit + host integration
Selection and approval	Exact retained hints use current files; summary never directly selects; stored-view digest and exact artifact approval; changed-target handling	Unit + controlled-tool integration

Claim	Required cases / source scopes	Tier
Picker/config/meter	Attached-only strategy option; full option set; setter interleaving; old flush/new pending; first sliding notice; largest actual dispatched request and no fabricated refused reading	Unit + real client
Final request guard	Equality/one-over at actual attempt boundary; trusted snapshot; all framing/material and one output reserve; incomplete finish blocked before plan approval	Unit + route evidence
Accounting	Exactly-once known/unknown/cancelled/stale attempts; divergent duplicate rejection; known subtotal retained; no re-debit on stored-plan application	Unit + real ledger
Product/evaluation split	Product result above former \$0.05 delivered; no dollar predicates/prompts/constructor requirements; finite count/time/failure retained; separate capped evaluation stops	Completed former strict-xfail assertions + integration

Use actual source scopes including tests/unit/context_engine, model_policy, acp, agent, usage, loops, optimus_gateway, tools, telemetry, retry, package-import tests and tests/integration/loops; use actual collection to select relevant names. Do not invent a passing test file or apply blanket xfail removal.

15.3 Per-route eligibility and D7 boundary evidence

Record exact model/provider/endpoint/quantization/reasoning, window ≥ 262144 and output caps $\geq$ role reserve, estimator algorithm/assets/version/digest, normalization/pre-tokenization and framing proof, enforced cap parameter and true finish semantics. Verify both actual request paths at exact and one-over complete capacity, with maximum permitted additional material and one output reserve.

The proposed named ADR-014 exception retains 524288-byte source/admission and its warning but permits readable OPEN refusal when full history cannot fit. Record operator acceptance before claiming that disposition. No source-floor reduction or capacity exemption. Public JSON/config inspection does not count tokens or install libraries; exact route-tokenizer dependencies remain deferred. Samples/family labels cannot establish conservative safety; unknown/expanding normalization remains ineligible. Verified $r=1$ substantially reduces usable exact tail/history in bytes, disclosed in the closure proposal.

15.4 Summary qualification

Use the existing tools/evaluate_context_summarizer.py and tests/fixtures/context_engine/calculator-constraints-v1.json. Step 1 summarizes turns 1-5; step 2 merges that actual summary with turns 6-7. Turns 8-9 are exact tail and cannot mask omission of either early constraints or the late correction. Evaluate the summary itself for Decimal, public calculate API, no eval/exec/network, negative inputs, HALF_EVEN as current, HALF_UP as superseded, both rounding examples, no invented approval/deletion and inert injection text. Protected approval projection must separately equal the host source.

The fixture output cap is **1200 tokens**, not the proposed production summary cap of 8192. Record this scope honestly. Require complete stop status and exact prompt/format/validator/fixture/model/endpoint/reasoning identities; a changed qualification key invalidates receipt reuse. Human review complements the deterministic lexical report. Two steps are a limited gate, not a statistical quality guarantee.

The current offline envelope reports two requests, input-token upper bounds 2333/2766 and up to two allowed provider attempts per request. Its old-price estimates and route blockers are not current paid eligibility. Qualification permits two logical requests, at most four physical attempts: one identical-route/payload resend only after NOT_SENT (including proven pre-connection timeout)/429. Require current complete input/framing/cost bounds and host non-retryable propagation; uncertain timeout/5xx/usage never retries. No alternative model recovery.

15.5 Real dependencies and editor

Task 13 uses independently authored acpx, the real Optimus process, real loopback Gateway, real TimeSeries-capable Redis and actual Zed rendering, under scoped credentials/TTY/service authority. The reviewed named test composition implements real server dependency injection and enabled GatewayModelPolicy, with HMAC launch approval/authenticated child-manifest hash binding. Positive offline proof uses a synthetic eligible profile; the real qualification profile remains ineligible. It cannot replace the client or claim production bootstrap activation. Use separate absent-engine and small Chat/Agent sessions; reuse one seeded attached session for large source, sliding, fault/recovery and cancellation, restoring compaction before maintenance. Maintenance requires real quality receipt/profile B; otherwise it is UNRUN. Vibhanshu supplies the named Zed prompts/approvals at the keyboard. Do not import old placeholder effects as real authority.

Prove mode/model/strategy option preservation, attached-only picker, sliding disclosure, separate storage/fallback/refusal notices, readable OPEN no-call refusal, truthful meter and absent-engine defaults. Exercise >512 KiB attached source, bounded fault/unavailable recovery and cancellation during maintenance. Record whether >floor history was generated through client traffic or explicitly seeded as synthetic canonical records; seeding is test setup, not proof of a natural conversation or durable resume.

Do not silently introduce a missing production seam to obtain evidence. Establish launch/tool/service prerequisites first and produce claim-specific non-passing rows when unavailable. A test seam's simulated unavailable response proves fault handling, not provider outage. Live cancellation can bill a started request; retain it.

15.6 Evidence package and closing verdict

Every claim records exact source/configuration/registry/launch identities, commands, dependency versions, selected test names, genuine individual exits, attempt receipts, sanitized conclusions and hashes. Preserve failures and seals. A unit fake cannot fill an acpx, Zed, Redis, Gateway or Phoenix live claim. Existing search/plugin and Phase 1 working-agent gates remain unchanged outside this Plan 12 package.

CP4 clears only when all its required measured-value, route, quality, migration and real-client obligations have named passing evidence or a separately recorded operator disposition. Coverage gates use full-suite data. The existing full WSL run before finalization remains unchanged; inventory that environment early and retain exact candidate, selection and exit. The two independent package floors are settled operator requirements. Evidence Handoff's 80% threshold is deferred in enforcement until revisited; interim coverage requires no new tests. Do not declare the broader Auto/reviewer/savings/loop/resume scopes complete. Deferred work retains its existing owners in the authoritative tracking venue, according to the actual Plan 11.29 cutover. Publication, merge and production activation remain separate decisions.

15.7 Completed offline evidence and remaining claims

Observed offline candidate, 2026-10-04: cf9bb9d (clean local CP4 branch). V1 capacity text/meter, V3 remaining-allowance preflight, Task 11 product/evaluation separation, named trusted test composition, tracked-source coverage gate and Task 12 boundaries pass offline verification. Windows: 6830 passed, 39 skipped, 111 deselected; WSL: 6641 passed, 228 skipped, 111 deselected; both exits 0. Production ENFORCEMENT_ACTIVE=False, trusted_snapshot() returns None, and the shipped claude-haiku-4.5 default are unchanged. Luna remains ineligible, Qwen remains unqualified, and paid qualification/real editor Task 13 remain UNRUN. See review-disposition.md and evidence-verification.json. This source review is not production activation or full CP4 closure.

The actual HostMaintenance callback and GatewaySummarizerCall boundary test includes summary-prompt/wire framing and one output reserve: equality admission, one-over INPUT_EXCEEDS_CAPACITY and zero upstream attempts on rejection. The final numeric proposal includes max_repacks=1. The refused case requires two repacks: one refuses after exactly one, while two fit. Positive profile capacity/finish proofs use synthetic test estimators and do not establish Luna eligibility.

The Windows and WSL run records name the exact clean candidate and full command; the sealed logs retain each environment's test-run context. WSL's Redis 6379 guard blocked its positive control and one existing reachability probe, which sent no Redis command. This is not evidence of arbitrary service isolation or Windows job parity.

Plan 9.88 historical claims use the frozen lane prompt version and retain all evidence checks. Its runner refuses new captures when the product prompt differs. Plan 9.87 keeps current-prompt validation. This preserves old evidence, not a new live acceptance.